

016: StepPPO-v3 设计 Review

2026-06-09

结论概览

StepPPO-v3 的总体方向是合理的。它不再继续调 StepPPO-v2 的混合权重，而是试图把 multi-turn agent 训练中的三个粒度对齐：环境 action 粒度、step-level credit assignment 粒度、以及 PPO clipping 粒度。相对 v2，v3 的核心改动是：删除 raw episode advantage，统一 shaped step reward，使用 pure step GAE advantage，默认 detach value backbone，并把 policy ratio 改成 response/action-level sequence geometric ratio。

该方向能直接回应现有日志里暴露的两个问题：一是 StepPPO-v2 的 reward 口径和 advantage 混合不够干净；二是 GiGPO value-aux detach=false 说明 shared-backbone value regression 本身就可能破坏 policy behavior。因此 v3-core 更像一个必要的诊断版本，而不是额外堆 trick 的复杂版本。

但是，当前 v3 设计在落地前仍有若干需要修正或明确的地方。最重要的是：配置字段需要和现有代码一致，`step_ppo_v3` 需要被 dispatch 识别，uid group whitening 的统计口径可能不完全合理，sequence geometric ratio 也不等价于完整 action probability ratio。

正向评价

- **reward 口径更干净。** v3 明确要求 policy advantage 和 value target 都来自同一套 shaped step reward，避免 v2 中 episode term 和 step term 分别看到不同 reward 语义。
- **value 干扰被隔离。** 默认 `detach_value_backbone=true` 是合理默认值。这样可以先验证 step credit assignment 是否有价值，而不是让 noisy value target 直接牵引 actor backbone。
- **MDP 粒度更一致。** WebShop 环境执行的是完整 textual action；v3 把 response/action 当成一个环境 action，并在 step 轴上做 GAE，比 token-as-step 的解释更自然。
- **diagnostics 更充分。** action ratio、value RMSE/MAE/EV、valid-vs-invalid 分层指标是必要的，否则很难判断失败来自 value、advantage 还是 policy ratio。
- **ablation 顺序合理。** v3-core、no-action-ratio、detach-false、lam-1、batch-norm-adv 这几个对照能较快定位主要因素。

必须先修的工程问题

1. adv estimator dispatch 不识别 `step_ppo_v3`

当前实现中 `ver1/trainer/ppo/step_ppo_algos.py::is_step_ppo_estimator` 只识别字符串尾部等于 `step_ppo` 的 estimator：

```
def is_step_ppo_estimator(adv_estimator) -> bool:
    return str(adv_estimator).lower().split(".")[1] == "step_ppo"
```

如果脚本直接设置:

```
algorithm.adv_estimator=step_ppo_v3
```

当前 trainer shim 不会调用 StepPPO advantage path。建议改为:

```
def is_step_ppo_estimator(adv_estimator) -> bool:
    name = str(adv_estimator).lower().split(".")[1]
    return name in {"step_ppo", "step_ppo_v3"}
```

同时在 compute_step_ppo_advantage_data 内根据 estimator 或 config namespace dispatch 到 v2/v3 两套逻辑。

2. policy loss 配置字段写法不匹配

现有配置结构是:

```
actor_rollout_ref:
  actor:
    policy_loss:
      loss_mode: "vanilla"
```

现有 actor 代码读取的是:

```
loss_mode = self.config.policy_loss.get("loss_mode", "vanilla")
```

因此 v3 脚本中不应写:

```
actor_rollout_ref.actor.policy_loss=action_level
```

这会把 dict 覆盖成字符串, 并导致 .get 失败。应改为:

```
actor_rollout_ref.actor.policy_loss.loss_mode=action_level
```

并在 StepWisePPOActor.update_policy 中显式支持 action_level。

3. 不能重复扣 invalid action penalty

当前主训练循环已经在 apply_invalid_action_penalty 中修改 token_level_scores, 并在已有 step_rewards 时同步扣除 step reward。v3 文档中的 build_shaped_step_rewards 如果再次执行:

```
rewards -= invalid_action_penalty_coef * (1 - is_action_valid)
```

就会双重惩罚 invalid action。

建议把 reward source 明确分成两种:

1. 如果 batch 已经存在 step_rewards 且 invalid penalty 已由 trainer 扣过, 则 v3 直接读取该字段。
2. 如果需要在 v3 helper 中构造 shaped reward, 则 trainer 侧不要提前扣, 或者 helper 读取 raw reward 字段并只扣一次。

最稳妥的实现是新增字段 raw_step_rewards 和 value_step_rewards, 避免同名字段被不同阶段原地修改后语义不清。

4. v3 需要保留 scalar step advantage 字段

现有 StepPPO 代码会把 final step advantage broadcast 成 token-shaped advantages, 同时保留 scalar step_advantages。v3 如果要想实现 action-level loss, 应该直接读取 scalar step_advantages; token-shaped advantages 只能作为兼容 fallback。

建议 actor update path 增加:

```
if loss_mode == "action_level":
    step_advantages = micro_data["step_advantages"]
    pg_loss, metrics = compute_policy_loss_action_level(...)
else:
    advantages = micro_data["advantages"]
```

这样可以避免 action-level loss 实际仍然通过 token broadcast 和 token aggregation 近似实现, 导致 metric 语义不清。

主要算法风险

1. uid whitening 可能混合不可比状态

v3 计划在同一 uid 内对所有 step samples 做 whitening。这个选择继承了 GiGPO/GRPO 的 group-relative 稳定性, 但在 step-level setting 中存在新问题: 同一个 task 的 step 0、step 1、step 2 状态分布并不相同, 早期探索动作和后期购买动作也不一定可比。

如果把同一个 uid 下所有 step 的 raw GAE advantage 一起 whitening, 可能出现两个副作用:

- 早期必要但 reward 延迟的动作被后期高 reward step 压成负 advantage。
- 不同 trajectory 长度的 step 数不同, 长 trajectory 对 uid statistics 贡献更多。

建议首轮至少记录 step-index 分层指标:

```
critic/step_adv_norm_mean_by_step_idx
critic/no_variance_group_ratio_by_step_idx
actor/action_clipfrac_by_step_idx
```

并把 ablation 从单纯 batch norm 扩展到:

```
advantage_norm in {uid, uid_step_idx, batch}
```

其中 uid_step_idx 表示同一 uid 且同一 step index 内做 normalization。

2. gamma=0.95, lambda=0.90 可能传播太短

v3 默认 $\gamma = 0.95$ 、 $\lambda = 0.90$, 有效 GAE 传播系数是:

$$\gamma\lambda = 0.855.$$

如果 WebShop reward 主要在 terminal 或 late-stage 出现, 这个设置可能显著削弱远期 credit。降低方差是好事, 但不能让 early-step credit 几乎消失。

建议 v3-core 的第一轮不要只跑 $\lambda = 0.90$, 而是至少保留:

$$\lambda \in \{0.90, 1.00\}$$

如果 $\lambda = 1.0$ 明显改善 valid action 或 validation, 再考虑细调 0.95。必要时也应测试 $\gamma = 1.0$, 因为当前 WebShop episode 长度较短, $\gamma = 1.0$ 的方差成本可能可控。

3. sequence geometric ratio 不是完整 action probability ratio

v3 使用：

$$\rho_i = \exp \left(\frac{1}{L_i} \sum_l m_{i,l} \Delta \log \pi_{i,l} \right).$$

它比 raw sequence ratio 更稳定，但它约束的是平均 per-token log-ratio，不是完整 response probability ratio：

$$\rho_i^{\text{sum}} = \exp \left(\sum_l m_{i,l} \Delta \log \pi_{i,l} \right).$$

因此长 response 的总概率变化仍可能很大，只是平均变化不大。这个设计可以作为工程折中，但文档里应明确它是 trust-region proxy，而不是严格 action probability ratio。

建议额外记录：

```
actor/action_log_ratio_sum_mean
actor/action_log_ratio_mean_mean
actor/action_ratio_geomean_mean
actor/action_ratio_sum_clipped_fraction
```

其中 sum ratio 不一定用于 loss，但用于发现长 response drift。

4. action-level clipping 可能过度保守或降低梯度强度

相比 token-level PPO，action-level geometric ratio 会让整个 response 共用一个 clip decision。优点是粒度一致；风险是某几个 token 的变化可能导致整段 action 被 clip，或者平均后掩盖局部 format token 的变化。

因此 v3_no_action_ratio 是必要 ablation。建议成功判据不要只看 validation score，还要比较：

- valid action ratio；
- response length clip ratio；
- action clipfrac 与 token pg clipfrac；
- old/new rollout probability diff；
- action format 错误样例。

5. terminal/truncation 语义需要明确

现有 compute_step_gae_advantage_return 对每条 trajectory 的最后一个 collected step 直接设 nonterminal=0。如果最后一步是环境自然 done，这是合理的；如果是 max_steps 截断，则严格来说应该 bootstrapping 到 next state value，或者至少记录为 truncated case。

v3 应在 rollout 中保存：

```
done_i
truncated_i
```

GAE 中使用：

$$\delta_i = r_i + \gamma(1 - \text{done}_i)V(s_{i+1}) - V(s_i),$$

并单独分析 truncation 比例。如果 WebShop 很多轨迹被 max step 截断，当前最后一步 return 会系统性偏低。

建议的实现顺序

1. 扩展 `is_step_ppo_estimator`, 让 `step_ppo_v3` 被 `trainer shim` 正确接管。
2. 新增 `algorithm.step_ppo_v3 namespace`, 但不要复用 `v2` 的 `algorithm.step_ppo` 字段。
3. 明确 `shaped reward` 只扣一次 `invalid penalty`, 保留 `raw_step_rewards` 和 `value_step_rewards`。
4. 新增 `compute_step_ppo_v3_advantage_return`, 输出 `scalar step_advantages`、`step_returns` 和 `diagnostics`。
5. 新增 `compute_policy_loss_action_level`, 不要只 `alias` 到 `GSPO`; 至少要返回 `action-level metrics`。
6. 修改 `StepWisePPOActor.update_policy`, 支持 `policy_loss.loss_mode=action_level` 和 `scalar advantage`。
7. 加 2GPU smoke, 先检查 `metric` 是否非 `NaN`, 再跑 4GPU full。

建议的 v3-core 配置修正版

```
algorithm.adv_estimator=step_ppo_v3
algorithm.gamma=0.95
algorithm.step_ppo_v3.reward_source=shaped_step
algorithm.step_ppo_v3.use_episode_advantage=false
algorithm.step_ppo_v3.step_gamma=0.95
algorithm.step_ppo_v3.step_lam=0.90
algorithm.step_ppo_v3.advantage_norm=uid
algorithm.step_ppo_v3.zero_no_variance_group=true
algorithm.step_ppo_v3.min_group_std=1.0e-6
algorithm.step_ppo_v3.policy_ratio=sequence_geometric
algorithm.step_ppo_v3.final_advantage_mode=pure_step
actor_rollout_ref.actor.policy_loss.loss_mode=action_level
actor_rollout_ref.actor.loss_agg_mode=seq-mean-token-mean
actor_rollout_ref.actor.value_head.detach_value_backbone=true
actor_rollout_ref.actor.value_head.value_loss_coef=0.03
actor_rollout_ref.actor.value_head.cliprange_value=0.5
```

Smoke Test Checklist

2GPU smoke 不建议直接看 `final score`, 应先确认以下指标:

1. `actor/action_ratio_mean`、`actor/action_clipfrac`、`actor/action_approx_kl` 正常出现。
2. `actor/value_rmse`、`actor/value_mae`、`actor/value_explained_variance` 非 `NaN`。
3. `detach=true` 下, `actor/grad_norm` 不因 `value loss` 明显放大。
4. `episode/valid_action_ratio` 不长期低于 0.5。
5. `response_length/clip_ratio` 不快速升到 `StepPPO-v2` 后期水平。
6. `critic/no_variance_group_ratio` 不长期接近 1。
7. `invalid vs valid action` 的 `advantage/value loss` 分层符合预期。

最终判断

StepPPO-v3 是一个值得实现的清晰诊断版本，但不应把它直接视为必然更强的算法。它最可能验证的命题是：当 reward 口径一致、value loss 不干扰 actor backbone、policy ratio 与 environment action 粒度一致时，step-level GAE 是否真的能提升 WebShop multi-turn credit assignment。

若 v3-core 仍明显弱于 GiGPO，优先排查三件事：uid whitening 是否消掉有效信号、 $\gamma\lambda$ 是否传播太短、action-level geometric ratio 是否过度保守。不要在这些问题未定位前引入更复杂 critic 或更多 auxiliary loss。